Conventional Commits 1.0.0

Guía simplificada en Español (Formato de una línea)

Especificación para agregar significado legible por humanos y máquinas a los mensajes de commit

1. Resumen

La especificación **Conventional Commits** es una convención ligera sobre los mensajes de commit. Proporciona un conjunto sencillo de reglas para crear un historial de commits explícito y ordenado, lo que facilita el trabajo en equipo y el uso de herramientas automatizadas. Esta convención se complementa con **SemVer** (Versionado Semántico), describiendo las nuevas funcionalidades, correcciones y cambios disruptivos (*breaking changes*) realizados en cada commit.

En su versión simplificada de una sola línea, el mensaje de commit se estructura así:

```
<type>[optional scope]:  <description>
```

Elementos estructurales del commit

1. `fix:` Un commit del *tipo* `fix` soluciona un error (*bug*) en el código (se correlaciona con una versión `PATCH` en SemVer).
2. `feat:` Un commit del *tipo* `feat` introduce una nueva funcionalidad o característica al código (se correlaciona con una versión `MINOR` en SemVer).
3. **Cambios disruptivos (`!`):** Agregar un signo de exclamación `!` justo después del tipo o alcance (por ejemplo, `feat!:` o `feat(api)!:`) indica que el commit introduce un cambio que rompe la compatibilidad con versiones anteriores (*Breaking Change*, correlacionado con `MAJOR` en SemVer). Puede formar parte de commits de cualquier *tipo*.
4. **Otros *tipos* permitidos:** Además de `fix:` y `feat:`, se recomiendan otros tipos basados en la convención de Angular:
   - `docs:` Cambios únicamente en la documentación (ej. `README.md`, Javadoc).
   - `refactor:` Cambio de código que no corrige un error ni agrega una funcionalidad.
   - `test:` Agregar pruebas faltantes (ej. JUnit, Mockito) o corregir pruebas existentes.
   - `style:` Cambios de formato (espacios, punto y coma, indentación) que no afectan el significado del código.
   - `chore:` Tareas de mantenimiento general o configuración que no modifican código de producción ni tests.
   - `build:` Cambios que afectan al sistema de compilación o dependencias externas (ej. Maven, Gradle, npm).
   - `perf:` Cambio de código que mejora el rendimiento.
   - `ci:` Cambios en archivos y scripts de configuración de Integración Continua.
5. **Alcance (`optional scope`):** Se puede proporcionar un alcance opcional entre paréntesis después del tipo para brindar información contextual sobre qué parte del código fue modificada. Ejemplo: `feat(parser):  add ability to parse arrays`.

2. Ejemplos (Formato de una línea)

- **Mensaje de commit simple (sin alcance):**

  ```
  docs: corregir ortografía en el archivo README
  ```

- **Mensaje de commit con alcance (*scope*):**

  ```
  feat(puestos): agregar validación al dar de alta un puesto
  ```

- **Mensaje de commit para corrección de errores con alcance:**

  ```
  fix(auth): evitar error nulo al autenticar usuario inexistente
  ```

- **Mensaje de commit con `!` para destacar un cambio disruptivo (*breaking change*):**

  ```
  feat!: enviar un email al cliente cuando se envía un producto
  ```

- **Mensaje de commit con alcance y `!` para un cambio disruptivo:**

  ```
  feat(api)!: cambiar formato de respuesta del endpoint de candidatos
  ```

3. Especificación (Reglas para commits de una línea)

1. Los commits **DEBEN** comenzar con un prefijo de tipo (`type`), que consiste en un sustantivo (`feat`, `fix`, etc.), seguido de un alcance (`scope`) **OPCIONAL**, un signo `!` **OPCIONAL**, y dos puntos seguidos de un espacio (`: `) **OBLIGATORIOS**.
2. El tipo `feat` **DEBE** usarse cuando un commit agrega una nueva funcionalidad a la aplicación o librería.
3. El tipo `fix` **DEBE** usarse cuando un commit representa la corrección de un error (*bug*) en la aplicación.
4. Un alcance (`scope`) **PUEDE** proporcionarse después del tipo. El alcance **DEBE** consistir en un sustantivo entre paréntesis que describa una sección del código, por ejemplo: `fix(dao):` o `feat(frontend):`.
5. Una descripción (`description`) **DEBE** seguir inmediatamente después de los dos puntos y el espacio (`: `) tras el prefijo de tipo/alcance. La descripción es un resumen breve de los cambios realizados en el código.
6. En el prefijo de tipo/alcance, los cambios disruptivos (*breaking changes*) **DEBEN** indicarse mediante un signo `!` inmediatamente antes de los dos puntos `:`. La descripción del commit se utiliza para describir el cambio.
7. Se **PUEDEN** utilizar otros tipos además de `feat` y `fix` en los mensajes de commit (por ejemplo, `docs: actualizar diagramas de secuencia`).
8. Las unidades de información que componen Conventional Commits **NO DEBEN** tratarse como sensibles a mayúsculas/minúsculas, aunque se recomienda mantener consistencia (usualmente todo en minúsculas).

4. ¿Por qué usar Conventional Commits?

- Generar registros de cambios (*CHANGELOGs*) de forma automática.
- Determinar automáticamente el incremento de versión semántica (según los tipos de commits realizados).

- Comunicar claramente la naturaleza de los cambios a los compañeros de equipo, docentes y otros colaboradores.
- Disparar procesos automatizados de compilación (*build*) y publicación.
- Facilitar que otras personas contribuyan al proyecto, permitiéndoles explorar un historial de commits mucho más estructurado y legible.

5. Preguntas Frecuentes (FAQ)

¿Cómo debo manejar los mensajes de commit en la fase inicial de desarrollo?
Se recomienda proceder como si ya hubieras lanzado el producto. Siempre hay alguien (incluso tus compañeros de grupo) usando el software, y querrán saber qué se agregó, qué se corrigió o qué cambió.

¿Los tipos en el título del commit van en mayúsculas o minúsculas?
Se puede usar cualquiera, pero es mejor ser consistente en todo el proyecto (la práctica estándar es usar **minúsculas**: `feat`, `fix`, `docs`).

¿Qué hago si el commit corresponde a más de un tipo de commit?
Siempre que sea posible, vuelve atrás y divide los cambios en múltiples commits más pequeños. Parte del beneficio de Conventional Commits es que nos impulsa a hacer commits y Pull Requests más organizados.

¿Esto no desalienta el desarrollo rápido y la iteración ágil?
Desalienta avanzar rápido de forma desorganizada. Te ayuda a poder avanzar rápido a largo plazo en proyectos con múltiples colaboradores.

¿Podría Conventional Commits limitar a los desarrolladores a pensar solo en los tipos provistos?
Conventional Commits nos anima a hacer más commits de ciertos tipos como correcciones (`fix`). Más allá de eso, su flexibilidad permite que tu equipo defina sus propios tipos si lo necesita.

¿Qué hago si accidentalmente uso el tipo de commit incorrecto?

- *Si usaste un tipo válido pero incorrecto (ej. `fix` en lugar de `feat`):* Antes de fusionar (*merge*) o entregar el código, se recomienda usar `git commit -amend` (si fue el último commit) o `git rebase -i` para editar el historial.
- *Si escribiste mal el tipo (ej. `feet` en lugar de `feat`):* En el peor de los casos, no es el fin del mundo si un commit no cumple la especificación; simplemente será ignorado por las herramientas automáticas.

¿Todos mis colaboradores necesitan usar la especificación Conventional Commits?
¡No necesariamente! Si utilizan un flujo basado en *Squash and Merge* en GitHub, quien revisa el Pull Request puede limpiar y unificar los mensajes de commit al momento de fusionar la rama.

¿Cómo se manejan los commits que revierten cambios (*revert*)?
Se recomienda usar el tipo `revert` seguido de la descripción del cambio que se está deshaciendo, por ejemplo: `revert(preguntas): deshacer baja lógica de preguntas`.